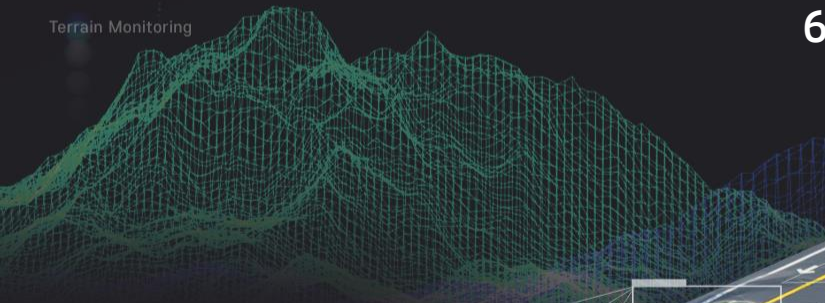


Image Recognition



Terrain Monitoring



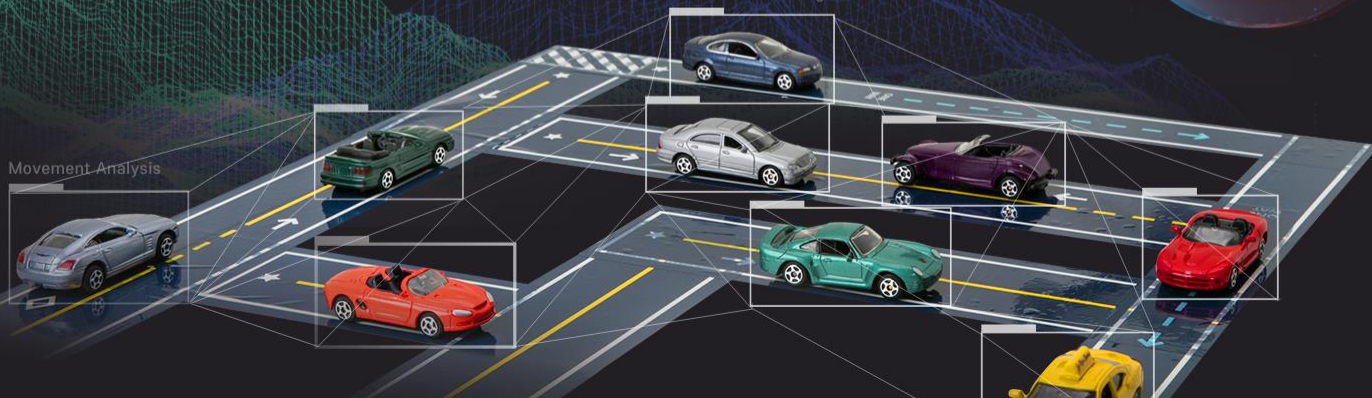
강원혁신플랫폼

컴퓨터 비전

6개의 클래스 분류

Autonomous Driving

Movement Analysis



Mars Exploration



이미지 다중 분류(Image Multi Classification)

6가지의 클래스로 구분된 사진 14,034장을 학습시켜서 새로운 사진을 입력했을 때 그것이 6가지의 분류 중 어디에 해당하는지 분류해 내는 모델을 만들어 보자.

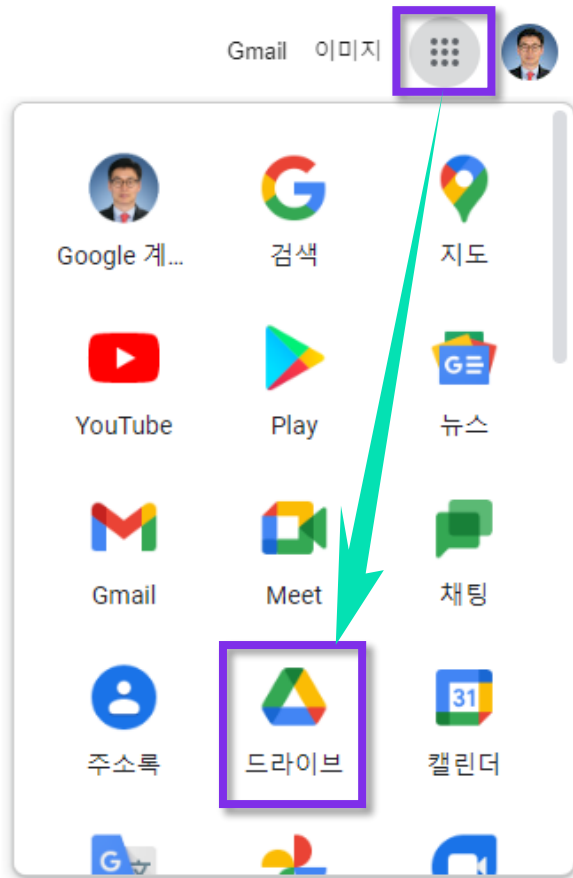
이미지 다중 분류(Image Multi Classification)

- 인공신경망(딥러닝)의 세부 알고리즘은 차차 배우기로 하고, 인텔(Intel)이 제공하는 6가지 이미지 분류의 전체과정과 실습을 통하여 학습해 보자.
- 6가지 클래스 인덱스는 아래 표와 같다.

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5

- 실습은 구글 Colab을 이용한다.

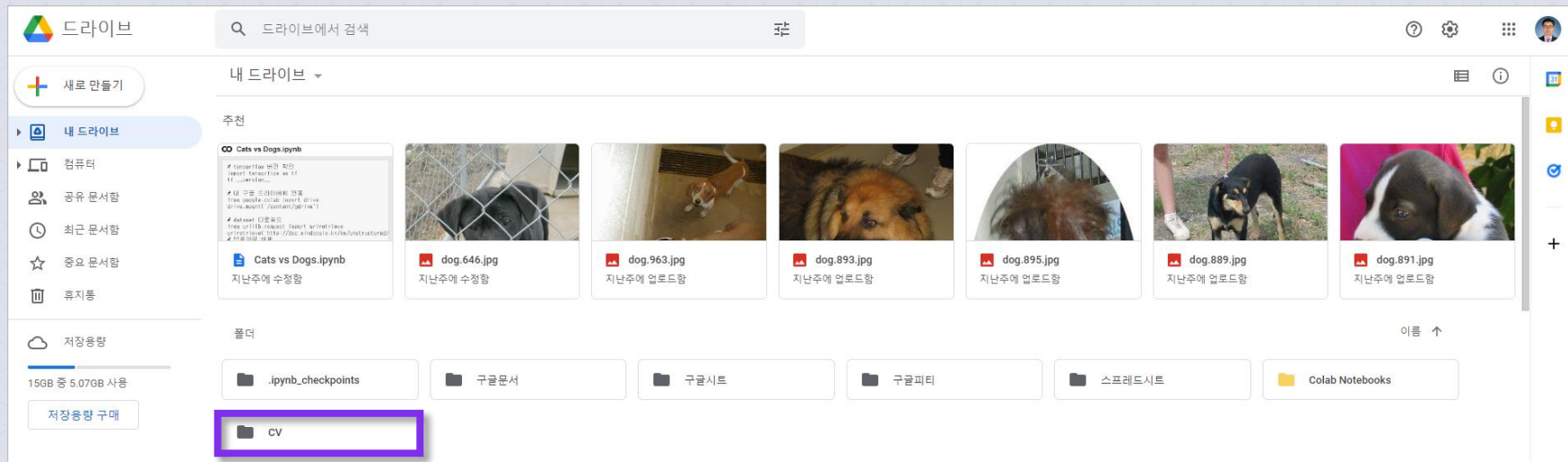
① 실습 파일 복사하기



- 구글 Colab을 이용하기 위해
구글 크롬 브라우저에서 그림과 같이
구글 드라이브를 클릭한다.

① 실습 파일 복사하기

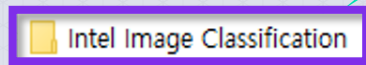
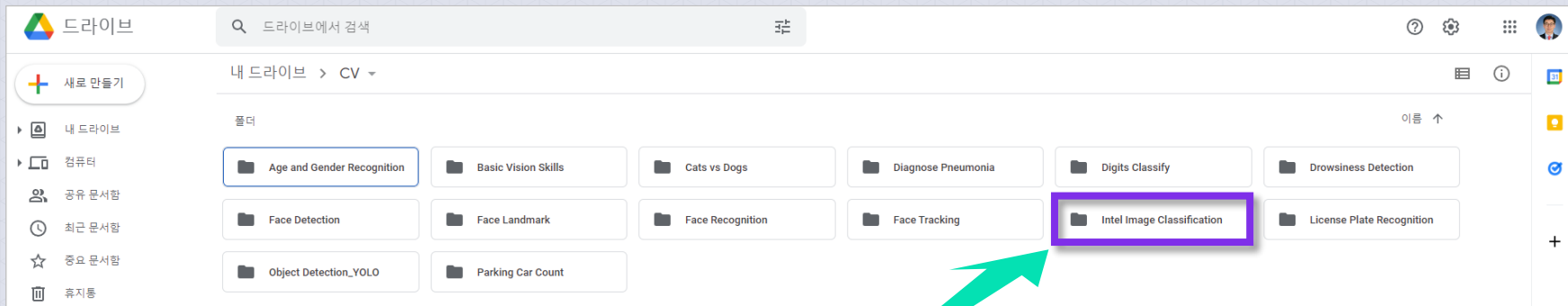
구글 드라이브에서 CV 폴더를 클릭한다.



CV 폴더는 개와 강아지 분류 문제에서 생성된 폴더이며,
CV 폴더가 없는 경우 직접 생성해도 됨(CV 폴더가 최상위 폴더)

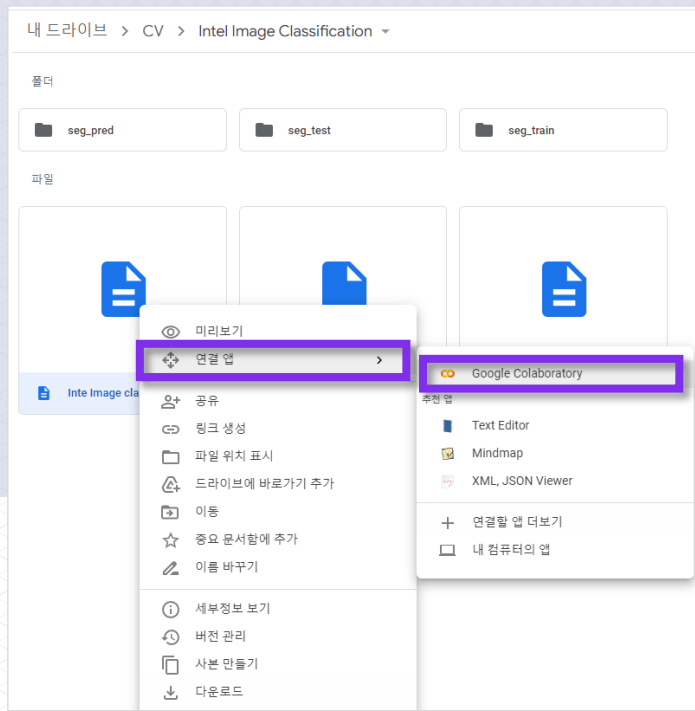
① 실습 파일 복사하기

- 구글 드라이브 > CV > Intel Image Classification 폴더를 클릭한다.
- 강의 실습 자료를 다운로드하여 [Intel Image Classification] 폴더를 구글 드라이브 CV 폴더 아래에 복사한다.



① 실습 파일 복사하기

- 아래의 그림의 [Intel Image Classification] 폴더에서 [Intel Image classification.ipynb](#)를 선택 후 오른쪽 마우스를 클릭하여 연결 앱 > [Google Colaboratory](#)을 클릭한다.



① 실습 파일 복사하기

- ☞ 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



② Tensorflow의 버전 확인하기

🟢 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.



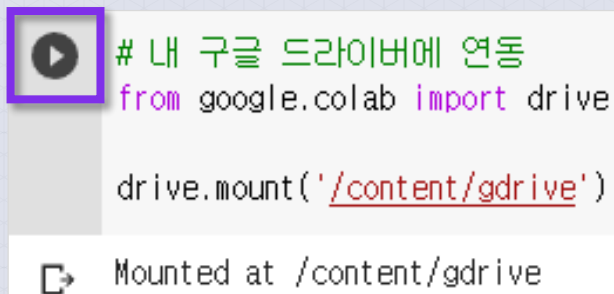
```
# tensorflow 버전 확인  
import tensorflow as tf  
tf.__version__
```

```
'2.8.2'
```

실행 결과 tensorflow 버전이 '2.8.2'인 것을 알 수 있다.

③ 구글 드라이브에 연동하기

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 내 구글 드라이버에 연동
from google.colab import drive

drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

③ 구글 드라이브에 연동하기

- 🟢 [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택한다.
- 🟢 gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요 **Google Drive에 연결**

Google 계정으로 로그인

계정 선택

Google Drive for desktop(으)로 이동

장원중
wjjang2040@gmail.com

② 다른 계정 사용

계속 진행하기 위해 Google에서 내 이름, 이메일 주소, 언어 환경 설정, 프로필 사진을 Google Drive for desktop과(와) 공유합니다. 앱을 사용하기 전에 Google Drive for desktop의 개인정보처리방침 및 서비스 약관을 검토하세요.

한국어 ▼ 도움말 개인정보처리방침 약관

구글 드라이브 접근 허용

Google 계정으로 로그인

Google Drive for desktop에서 내 Google 계정에 액세스하려고 합니다

wjjang2040@gmail.com

어떻게 하면 Google Drive for desktop에서 내 작업을 할 수 있습니까?

- Google Drive 파일 보기, 수정, 생성, 삭제 ①
- Google 포토의 사진, 동영상, 앨범을 봅니다. ①
- 모바일 클라이언트로 구성 및 실행 정보 가져오기 ①
- 프로필 및 연락처와 같은 Google 사용자 정보 조회 ①
- Google Drive 파일 작업 기록 조회 ①
- Google Drive 문서 보기, 수정, 생성, 삭제 ①

Google Drive for desktop 앱을 실행할 수 있는지 확인

명확한 정보가 이 사이트 또는 앱과 공유될 수 있습니다. 언제든지 Google 계정에서 액세스 권한을 확인하고 삭제할 수 있습니다.

Google이 데이터를 안전하게 공유하는 방법을 알아보세요.

Google Drive for desktop이 개인정보처리방침 및 서비스 약관을 확인하세요.

취소 **이동**

한국어 ▼ 도움말 개인정보처리방침 약관

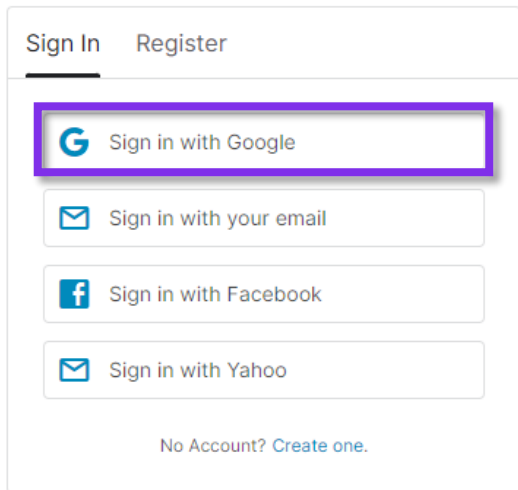
④ Kaggle에 연동하여 데이터 셋 다운받기

- 1 ▶ 모델의 학습과 예측에 사용할 intel-image-classification 데이터 셋이 Kaggle에 있으므로, 구글 Calab에서 Kaggle에 연동하여 데이터 셋을 다운로드 받는다.
- 2 ▶ (선택사항) Kaggle에서 데이터 셋을 다운 받기 위해서는 Kaggle에 우선 회원가입이 되어 있어야 한다.
- 3 ▶ intel-image-classification Kaggle에서 직접 다운로드 받는 절차는 반드시 필요하다.
 - 🔄 인텔이 제공하는 6개의 이미지 분류 데이터 셋을 파일첨부하여 제공하기 어렵기 때문임
 - 🔄 여기서 Kaggle 회원가입 절차는 생략한다.

④ Kaggle에 연동하여 데이터 셋 다운받기

kaggle

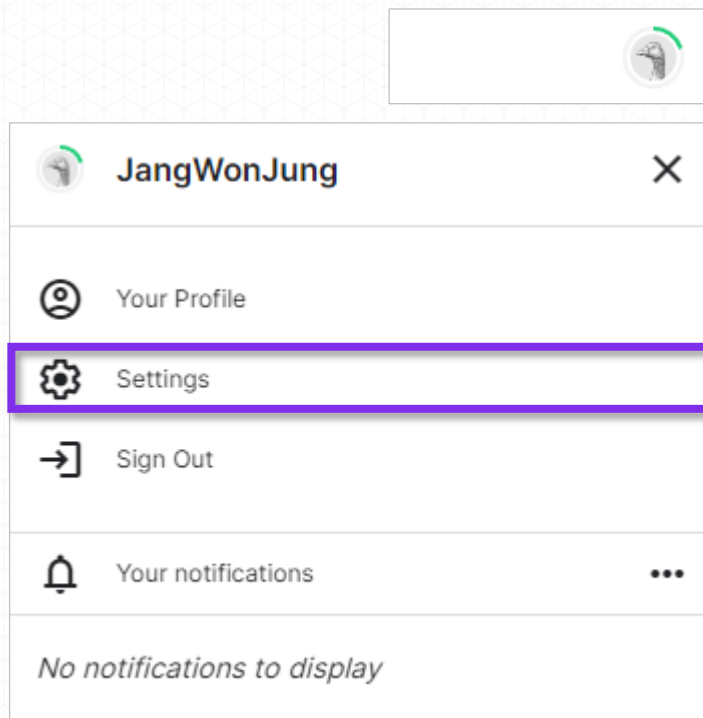
- 🔄 Kaggle에 연동하기 위해서 API Token을 받아와야 하는데 일단 kaggle.com로 접속한다.
- 🔄 그림과 같은 창이 뜨면 로그인을 한다.



The image shows the Kaggle login/register interface. At the top, there are two tabs: 'Sign In' and 'Register'. Below the tabs, there are four buttons for signing in: 'Sign in with Google' (highlighted with a red border), 'Sign in with your email', 'Sign in with Facebook', and 'Sign in with Yahoo'. At the bottom, there is a link that says 'No Account? Create one.'

④ Kaggle에 연동하여 데이터 셋 다운받기

- 로그인 하면 우측 상단에 오리 아이콘을 클릭하고, Settings를 선택한다.



④ Kaggle에 연동하여 데이터 셋 다운받기

- 아래 화면의 중간쯤 스크롤을 내리면 새로운 API를 생성할 수 있는 [Create New Token] 버튼이 있다.
- 이 버튼을 누르면 kaggle.json 파일이 다운로드 된다.

Settings

Control over your Kaggle account and all communications

Account Notifications

Your email address
wjiang2040@gmail.com
Change email

Your username
jangwonjung (not editable)

Your account number
7160486

Phone verification

Your account is not verified. Verifying your account with a phone number allows you to do more on Kaggle, and helps prevent spam and other abuse.

Phone verify

API

Using Kaggle's beta API, you can interact with Competitions and Datasets to download data, make submissions, and more via the command line. [Read the docs](#)

☒ Ensure kaggle.json is in the location ~/.kaggle/kaggle.json to use the API.

Dismiss

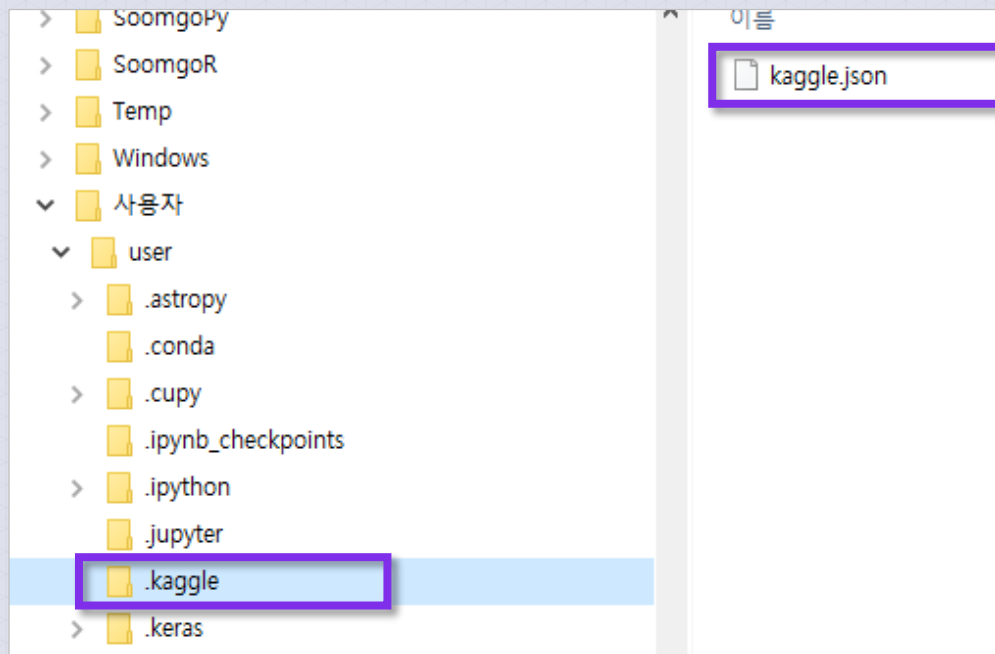
Create New Token

Expire Token

④ Kaggle에 연동하여 데이터 셋 다운받기

아나콘다(Anaconda)를 사용하는 경우

- ➡ [사용자 계정] 폴더 > .kaggle 폴더 아래에 kaggle.json 파일을 복사한다.
- ➡ 아래의 경우 사용자 계정이 'user'인 경우의 예시이다.



④ Kaggle에 연동하여 데이터 셋 다운받기

구글 Colab인 경우

- ➡ Kaggle API(kaggle 라이브러리)를 설치한다.
- ➡ 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.



1. 먼저 본인의 Kaggle 계정에서 API Token 을 다운받습니다.(kaggle.json)

2. Kaggle API를 설치합니다.
!pip install kaggle



Looking in indexes: <https://pypi.org/simple>, <https://us-python.pkg.dev/colab-wheels/public/simple/>

Requirement already satisfied: kaggle in /usr/local/lib/python3.7/dist-packages (1.5.12)

Requirement already satisfied: certifi in /usr/local/lib/python3.7/dist-packages (from kaggle) (2022.6.15)

Requirement already satisfied: python-slugify in /usr/local/lib/python3.7/dist-packages (from kaggle) (6.1.2)

Requirement already satisfied: requests in /usr/local/lib/python3.7/dist-packages (from kaggle) (2.23.0)

Requirement already satisfied: python-dateutil in /usr/local/lib/python3.7/dist-packages (from kaggle) (2.8.2)

Requirement already satisfied: urllib3 in /usr/local/lib/python3.7/dist-packages (from kaggle) (1.24.3)

Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.7/dist-packages (from kaggle) (1.15.0)

Requirement already satisfied: tqdm in /usr/local/lib/python3.7/dist-packages (from kaggle) (4.64.0)

Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.7/dist-packages (from python-slugify->kaggle) (1.3)

Requirement already satisfied: chardet<4,>=3.0.2 in /usr/local/lib/python3.7/dist-packages (from requests->kaggle) (3.0.4)

Requirement already satisfied: idna<3,>=2.5 in /usr/local/lib/python3.7/dist-packages (from requests->kaggle) (2.10)

④ Kaggle에 연동하여 데이터 셋 다운받기

- `kaggle.json` 파일을 VM에 업로드 한다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.
- 구글 Colab 노트북 화면에서 `kaggle.json` 파일을 VM에 업로드 하는 경우 아래 화면의 [파일 선택]을 클릭하여 `kaggle.json` 파일을 업로드 해 준다.



3.kaggle.json 파일을 VM에 업로드 합니다.

```
from google.colab import files  
files.upload()
```



파일 선택 선택된 파일 없음

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving kaggle.json to kaggle.json

```
{'kaggle.json': b'{"username": "jangwonjung", "key": "14b871b16558c2d7def8951e31896828"}'}
```

④ Kaggle에 연동하여 데이터 셋 다운받기

- `kaggle.json` 파일이 제대로 업로드 되었는지 확인한다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



4.파일이 제대로 업로드 됐는지 확인

```
!!ls -lha kaggle.json
```



```
kaggle.json
```

④ Kaggle에 연동하여 데이터 셋 다운받기

- ➡ Kaggle API가 Token을 사용하게 하기 위해 kaggle.json 파일을 ~/.kaggle 폴더로 이동시킨다.
- ➡ 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
1 # 5.Kaggle API 가 Token을 사용하게 하기 위해 kaggle.json 파일을 ~/.kaggle로 이동 시킵니다.  
2 !mkdir -p ~/.kaggle  
3 !mv kaggle.json ~/.kaggle/
```

④ Kaggle에 연동하여 데이터 셋 다운받기

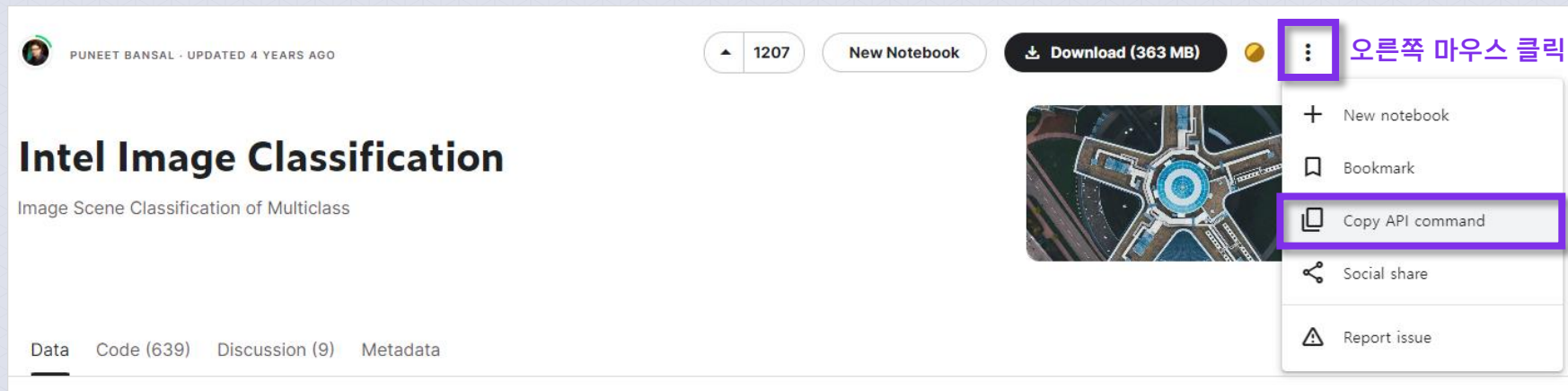
- ➡ Permission Warning이 일어나지 않도록 권한을 부여한다.
- ➡ 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
1 # 6. Permission Warning 이 일어나지 않도록 권한 부여  
2 !chmod 600 ~/.kaggle/kaggle.json
```

④ Kaggle에 연동하여 데이터 셋 다운받기

- ➡ Kaggle의 **intel-image-classification** 데이터 셋을 **다운로드** 받기 위해 [https://www.kaggle.com/datasets/puneet6060/intel-image-classification]로 **접속**한다.
- ➡ 위 URL로 접속하면 아래 화면이 출력되고, 이 화면에서 **Copy API command**를 **선택**한다.



④ Kaggle에 연동하여 데이터 셋 다운받기

- [Copy API command]를 [Ctrl + v]로 셀에 붙여 넣은 다음 명령어를 실행하면 실행하면 **intel-image-classification** 데이터 셋이 **다운로드** 된다.
- 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.



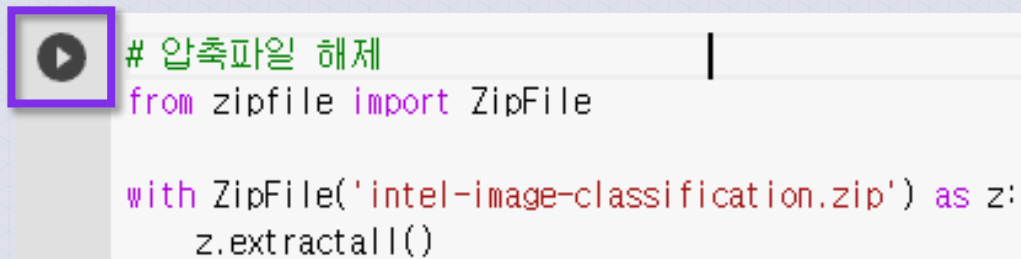
```
# dataset URL : https://www.kaggle.com/puneet6060/intel-image-classification  
# 7.Kaggle 에서 intel-image-classification dataset을 다운 받기 위해 API Command 를 복사해서 가져옵니다.  
!kaggle datasets download -d puneet6060/intel-image-classification
```

```
Downloading intel-image-classification.zip to /content  
95% 329M/346M [00:11<00:00, 38.0MB/s]  
100% 346M/346M [00:11<00:00, 31.3MB/s]
```

실행 결과 '**intel-image-classification.zip**' 이름으로 VM에 **다운로드** 된다.

⑤ 다운받은 데이터 셋을 구글 드라이브에 복사하기

- 다운받은 zip 파일을 압축해제 한다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 압축파일 해제
from zipfile import ZipFile

with ZipFile('intel-image-classification.zip') as z:
    z.extractall()
```

실행 결과 VM에 seg_pred, seg_test, seg_train 폴더가 생성된다.

⑤ 다운받은 데이터 셋을 구글 드라이브에 복사하기

- intel-image-classification 데이터 셋을 내 구글 드라이브로 이동시킨다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



dataset을 내 구글 드라이브로 이동시킴

```
!mv seg_pred /content/gdrive/My Drive/CV/Intel Image Classification/  
!mv seg_test /content/gdrive/My Drive/CV/Intel Image Classification/  
!mv seg_train /content/gdrive/My Drive/CV/Intel Image Classification/
```

- 실행결과 아래의 폴더에 intel-image-classification 데이터 셋을 복사한다.
- Intel Image Classification > seg_pred
> seq_test
> seq_train

⑤ 다운받은 데이터 셋을 구글 드라이브에 복사하기

- 다운로드 받은 'intel-image-classification.zip' 파일을 삭제한다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



다운로드 받았던 파일 제거

!rm intel-image-classification.zip

⑤ 다운받은 데이터 셋을 구글 드라이브에 복사하기

- 구글 드라이브로 이동된 intel-image-classification 데이터 셋을 확인한다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.

```
# 구글 드라이브로 이동된 dataset 확인
!ls /content/gdrive/My Drive/Colab Image Classification/seg_train/seg_train

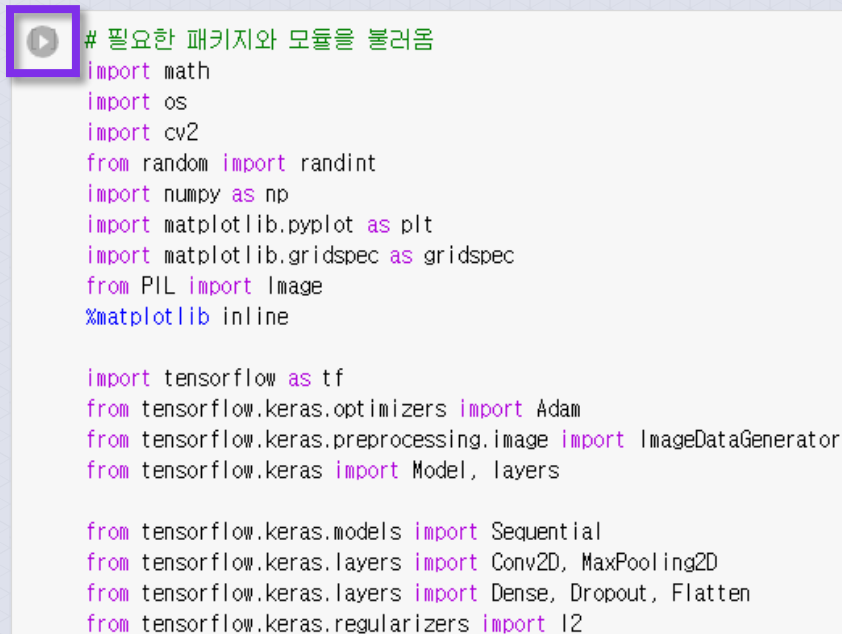
buildings forest glacier mountain sea street
```

- 실행결과 아래의 폴더에 intel-image-classification 데이터 셋이 있는 것을 알 수 있다.
- Intel Image Classification > seg_train > seq_train > buildings
> forest
> glacier
> mountain
> sea
> street

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5

⑥ 필요한 패키지와 모델을 불러오기

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 필요한 패키지와 모듈을 불러옴
import math
import os
import cv2
from random import randint
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from PIL import Image
%matplotlib inline

import tensorflow as tf
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import Model, layers

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.regularizers import l2
```

학습에 필요한 패키지를 읽어온다(메모리에 로딩한다).

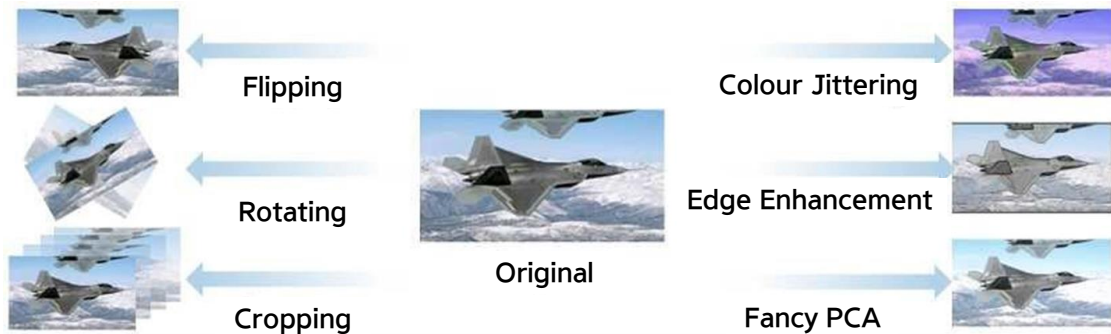
⊕ 6개의 클래스 분류(학습)

다음은 ImageDataGenerator 클래스를 통해 `train_datagen` 인스턴스를 만든다.

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨

```
# ImageDataGenerator 클래스를 통해 train_datagen 인스턴스를 만듭니다.  
train_datagen = ImageDataGenerator(  
    zoom_range=0.2,      # 20%까지 확대  
    rotation_range=10,   # 10도까지 회전  
    horizontal_flip=True, # 좌우 뒤집기  
    rescale=1./255)     # 픽셀 값을 0~1 범위로 변환
```

- tensorflow.keras의 ImageDataGenerator 클래스는 데이터 증강(Data Augmentation)을 지원



여러 가지 데이터 증강 기법



■ 이미지 회전과 무작위 변환을 적용해 훈련 데이터의 다양성을 증가시키는 기술

- ➡ 데이터의 양을 늘려 과적합 방지
- ➡ 훈련 데이터에만 적용
- ➡ 다양한 데이터 변형 상황을 대비

예 숫자 6, 9 이미지와 같은 데이터일 경우 오히려 학습에 방해가 될 수 있으므로 데이터의 정보를 파악하고 적당히 변형시켜야 함(회전 10~20% 정도)

이미지 증강

예시

크기 조절, 회전, 밝기, 색깔 변경, 잘라내기



■ 데이터 증강(Data Augmentation)은 왜 필요할까?

1 ▶ 과적합(오버피팅) 문제

- ❶ 이미지 분류 CNN 모델도 정형 데이터를 예측하는 일반적인 머신러닝 모델과 마찬가지로, 극복해야 할 문제
- ❷ CNN 모델의 성능을 높이면서 오버피팅을 극복할 수 있는 가장 좋고 근본적인 해결책은 학습 데이터의 다양성을 늘리는 것임

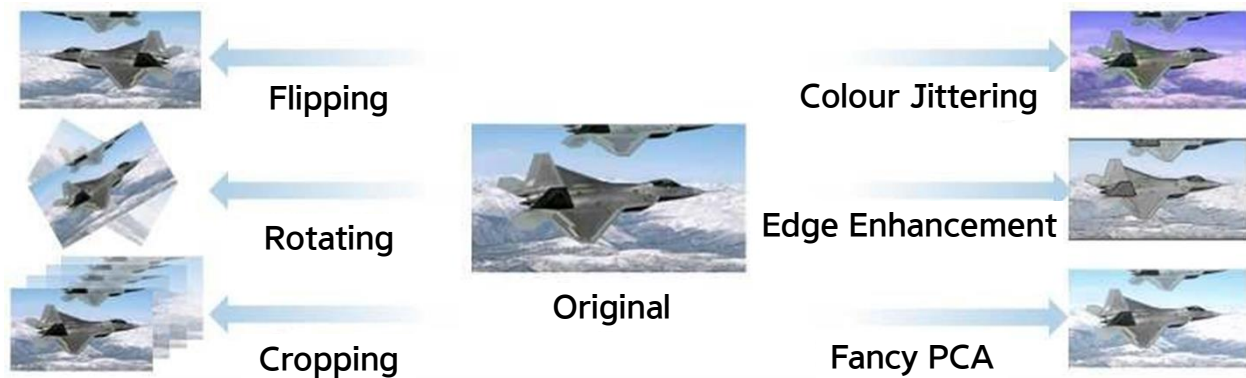


■ 데이터 증강(Data Augmentation)은 왜 필요할까?

2 이미지 데이터 추가

- ❶ 학습 데이터의 개수를 직접 늘리면 더할 나위 없이 좋지만 이미지 데이터를 추가로 구하는 것은 사람의 손을 필히 거쳐야 하며 비용도 시간도 많이 드는 일임
- ❷ 비용과 시간도 적게 소요하면서 학습 데이터의 다양성을 늘릴 수 있는 방법으로 고안된 것이 **데이터 증강 기법**
 - └ 하나의 원본 이미지를 다양한 버전으로 만들어 학습시키는 것

데이터 증강(Data Augmentation)



여러 가지 데이터 증강 기법

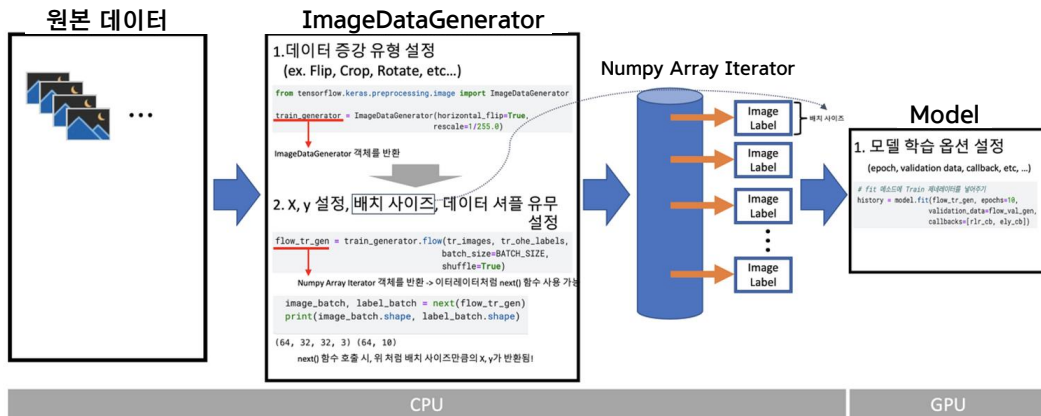
대표적 데이터 증강 기법

- Flipping : 원본 이미지를 수평 또는 수직으로 반전시키는 방법
- Rotating : 회전시키는 방법
- Cropping : 일부를 자르는 방법
- Colour Jittering : RGB 채널 순서를 바꾸거나 픽셀값을 변경하여 원본 이미지의 색조를 변경시키는 방법

텐스플로에서는 원본 이미지 증강 기법을 tensorflow.keras의 **ImageDataGenerator** 클래스가 대신함

ImageDataGenerator 클래스의 역할

- 원본 이미지 데이터를 증강
- 텐스플로 내에서 하나의 데이터 파이프라인을 구축
- Tensorflow.keras의 preprocessing 모듈에 존재



ImageDataGenerator 클래스의 과정

※ ImageDataGeneratorExam.ipynb
프로그램 실습을 통해
좀 더 깊이 있게 학습해 보자.

⊕ 6개의 클래스 분류(학습)

좀 더 구체적으로 ImageDataGenerator 과정을 정리해 보면 다음과 같다.

1 ImageDataGenerator 객체를 생성하면서 데이터 증강을 수행할 유형들을 지정함

```
1 # ImageDataGenerator 클래스를 통해 train_datagen 인스턴스를 만듭니다.  
2 train_datagen = ImageDataGenerator(  
3     zoom_range=0.2,      # 20%까지 확대  
4     rotation_range=10,   # 10도까지 회전  
5     horizontal_flip=True, # 좌우 뒤집기  
6     rescale=1./255)      # 픽셀 값을 0~1 범위로 변환
```

🔄 위 코드와 같이 다양한 데이터 증강 기법을 설정하는 것을 볼 수 있음

⊕ 6개의 클래스 분류(학습)

좀 더 구체적으로 ImageDataGenerator 과정을 정리해 보면 다음과 같다.

2 1단계에서 생성한 ImageDataGenerator 객체를 생성한 후 **훈련 데이터**에서 **배치 크기** 만큼 이미지를 만들어 냄

- ➡ 학습 이미지(X), 정답 레이블(y) 이미지의 **배치 사이즈** 설정
- ➡ **데이터 셔플** 여부 지정
- ➡ 반환되는 **레이블 배열의 유형** 지정
- ➡ Numpy **Array Iterator**(iterator 안의 데이터를 하나씩 호출)

```
1 # train_datagen 인스턴스를 통해 train dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 train_generator 객체를 생성한다
2 train_generator = train_datagen.flow_from_directory(
3     'gdrive/My Drive/CV/Intel Image Classification/seg_train/seg_train',
4     batch_size=100,      # Size of the batches of data (default: 32).
5     class_mode='sparse', # Determines the type of label arrays that are returned. "sparse" will be 1D integer labels
6     target_size=(150, 150)) # 이미지의 크기 재정의
```

- ➡ 위와 같이 설정하는 이유는 **이미지 사이즈가 커질수록 한번에 이미지**를 CPU, GPU에 올릴 수 없기 때문에 **배치 사이즈 크기 설정** 같은 처리를 미리 수행함

⊕ 6개의 클래스 분류(학습)

좀 더 구체적으로 ImageDataGenerator 과정을 정리해 보면 다음과 같다.

3 모델을 학습시키기 위해 필요한 하이퍼파라미터를 설정함

- ➊ 2단계에서 만들어진 iterator를 생성된 모델의 메서드(fit 또는 fit_generator)에 입력
- ➋ 에폭(반복 학습 횟수) 설정
- ➌ 콜백함수(모델이 더 이상 학습을 못할 경우 종료) 설정

```
1  # model training(모델을 학습시킵니다)
2  history = model.fit_generator(generator=train_generator,
3                               steps_per_epoch=140,
4                               epochs=40,
5                               validation_data=test_generator,
6                               validation_steps=30)
```


⊕ 6개의 클래스 분류(학습)

train_datagen 인스턴스를 통해 **훈련 데이터** 셋에서 한번에 **batch_size** 만큼 **이미지**를 만들어내는 **train_generator** 객체를 **생성**한다.

⊕ train_datagen 인스턴스는 ImageDataGenerator 클래스의 객체

```
# train_datagen 인스턴스를 통해 train dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 train_generator 객체를 생성한다
train_generator = train_datagen.flow_from_directory(
    'gdrive/My Drive/CV/Intel Image Classification/seg_train/seg_train',
    batch_size=100,      # Size of the batches of data (default: 32).
    class_mode='sparse', # Determines the type of label arrays that are returned. "sparse" will be 1D integer labels
    target_size=(150, 150)) # 이미지의 크기 재정의
```

Found 14034 images belonging to 6 classes.

- ⊕ flow_from_directory() 함수는 **directory** 경로로부터 **데이터프레임**을 가져오고, 증강(Augmented) 또는 정규화(Normalized)된 **데이터 배치**를 **생성**함
- ⊕ 위 코드에서 **훈련용 데이터** 셋에서 **batch_size=100** 만큼 이미지로 **train_generator** 객체가 **만들어진 것**을 알 수 있음
- ⊕ 실행 결과 훈련 데이터 셋으로 **6개의 클래스**로 구성된 **14,034장의 이미지**가 있음을 알 수 있음

⊕ 6개의 클래스 분류(학습)

ImageDataGenerator 클래스를 통해 `test_datagen` 인스턴스를 만든다.

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨



```
1 # ImageDataGenerator 클래스를 통해 test_datagen 인스턴스를 만듭니다.  
2 test_datagen = ImageDataGenerator(rescale=1./255) # 픽셀 값을 0~1 범위로 변환
```

- 테스트 데이터 셋에는 데이터 정규화(0~255 → 0~1)만 적용함

⊕ 6개의 클래스 분류(학습)

train_datagen 인스턴스를 통해 테스트 데이터 셋에서 한번에 batch_size 만큼 이미지를 만들어내는 train_generator 객체를 생성한다.

⊕ train_datagen 인스턴스는 ImageDataGenerator 클래스의 객체

```
# ImageDataGenerator를 통해 validation dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 test_generator 객체를 생성한다.  
test_generator = test_datagen.flow_from_directory(  
    'gdrive/My Drive/CV/Intel Image Classification/seg_test/seg_test',  
    batch_size=100,  
    class_mode='sparse',  
    target_size=(150, 150))
```

Found 3000 images belonging to 6 classes.

- ⊕ flow_from_directory() 함수는 directory 경로로부터 데이터프레임을 가져오고, 증강(Augmented) 또는 정규화(Normalized)된 데이터 배치를 생성함
- ⊕ 위 코드에서 테스트 데이터 셋에서 batch_size=100 만큼 이미지로 train_generator 객체가 만들어진 것을 알 수 있음
- ⊕ 실행 결과 훈련 데이터 셋으로 6개의 클래스로 구성된 3,000장의 이미지가 있음을 알 수 있음

⊕ 6개의 클래스 분류(학습)

test_generator 객체가 만들어 내는 이미지의 클래스 인덱스를 확인해 보자.

```
# test_generator 가 만들어 내는 이미지의 클래스 인덱스가 무엇인지 확인해 본다.  
print(test_generator.class_indices)
```

```
{'buildings': 0, 'forest': 1, 'glacier': 2, 'mountain': 3, 'sea': 4, 'street': 5}
```

- 실행 결과는 0:빌딩, 1:숲, 2:빙하, 3:산, 4:바다, 5:거리임을 알 수 있음

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5

- 위 코드에서 class_indices 객체 변수의 자료구조는 딕셔너리인 것을 알 수 있음

⊕ 6개의 클래스 분류(학습)

다음은 레이블 명(label name)을 찾아오는 findKey() 함수를 정의한다.

```
▶ # label name 을 찾아 오는 함수를 정의한다.  
def findKey(indices, search_value):  
    for key, value in indices.items():  
        if(value == search_value):  
            return key  
    return -1
```

- ⦿ 위 코드에서 findKey() 함수는 indices 객체(딕셔너리)의 value와 search_value의 값이 동일한 경우, indices 객체의 key 값을 반환함

예) indices = {'buildings':0, 'forest':1, 'glacier':2, 'mountain':3, 'sea':4, 'street':5},
search_value = 0인 경우, findKey() 함수의 반환값은 'building'이 됨

⊕ 6개의 클래스 분류(학습)

다음은 훈련용 데이터 셋에서 샘플 데이터(16개 이미지)를 출력해 보자.

```
▶ # training dataset 에서 샘플 데이터를 display 해본다
for X_batch, y_batch in train_datagen.flow_from_directory('gdrive/My Drive/CV/Intel Image Classification/seg_train/seg_train',
                                                         batch_size=100, class_mode='sparse', target_size=(150, 150)):

    plt.figure(figsize=(20,20))
    # create a grid of 4x4 images
    for i in range(0, 16):
        ax = plt.subplot(4, 4, i+1)
        ax.set_title(findKey(train_generator.class_indices, y_batch[i])) # 이미지의 label name 을 찾아온다
        plt.imshow((X_batch[i].reshape(150, 150, 3)*255).astype(np.uint8)) # train 샘플 이미지를 보여준다.
    # show the plot
    plt.show()
    break
```

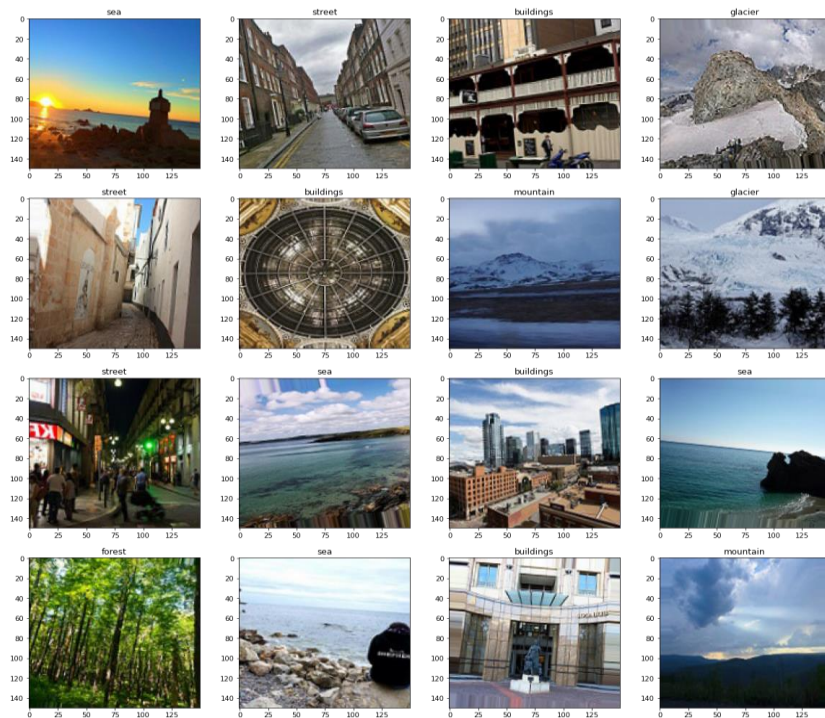
⊕ 위 코드에서 16장의 이미지를 출력하고 반복문을 종료한 것을 알 수 있음

- 위 코드의 첫 번째 반복문은 훈련용 데이터를 100장 단위로 묶어서 읽어옴
- 두 번째 반복문은 16번 반복하면서 첫 번째 반복문에서 읽어온 16장 이미지 각각의 레이블을 이미지 제목으로 (150, 150, 3) 형상으로 화면에 출력함
- 두 번째 반복문이 끝나면 16장의 이미지를 화면에 출력하고 종료함

6개의 클래스 분류(학습)

다음은 훈련용 데이터 셋에서 샘플 데이터(16개 이미지)를 출력해 보자.

실행 결과 16장의 이미지는 아래 그림과 같음



⊕ 6개의 클래스 분류(학습)

다음은 인공지능망 네트워크를 구성하여 이미지 분류를 위한 모델을 만든다.

- ⊕ CNN, Fully connected neural network, Relu, Sigmoid, MaxPooling, Cross entropy error, RMSprop, Accuracy, Dropout 알고리즘을 이용함

```
# create model(모델을 만듭니다)
model = Sequential()
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu', input_shape=(150, 150, 3), name="conv1"))
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu', name="conv2"))
model.add(MaxPooling2D(pool_size=(3, 3), name="maxpool1"))
model.add(Conv2D(128, kernel_size=(3, 3), activation='relu', name="conv3"))
model.add(Conv2D(128, kernel_size=(3, 3), activation='relu', name="conv4"))
model.add(MaxPooling2D(pool_size=(3, 3), name="maxpool2"))
model.add(Conv2D(256, kernel_size=(3, 3), activation='relu', name="conv5"))
model.add(Conv2D(256, kernel_size=(3, 3), activation='relu', name="conv6"))
model.add(MaxPooling2D(pool_size=(3, 3), name="maxpool3"))
model.add(Flatten())
model.add(Dense(1024, activation='relu', kernel_regularizer=l2(0.001), bias_regularizer=l2(0.001)))
model.add(Dense(256, activation='relu', kernel_regularizer=l2(0.001), bias_regularizer=l2(0.001)))
model.add(Dropout(0.25))
model.add(Dense(6, activation='softmax'))

model.compile(loss='sparse_categorical_crossentropy',
              optimizer=Adam(learning_rate=0.0001),
              metrics=['accuracy'])

model.summary()
```

옆 코드와 같이
9층 인공지능망 모델을 생성

출력층 활성화 함수 : 소프트맥스

6개의 클래스 분류(학습)

다음은 인공지능망 네트워크를 구성하여 이미지 분류를 위한 모델을 만든다.

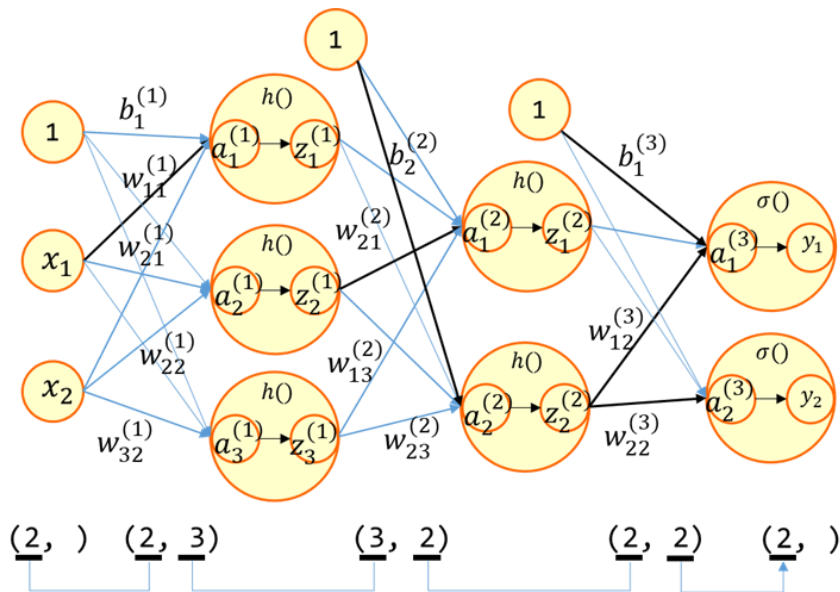
인공지능망 네트워크 구성 결과

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 148, 148, 64)	1792
conv2 (Conv2D)	(None, 146, 146, 64)	36928
maxpool1 (MaxPooling2D)	(None, 48, 48, 64)	0
conv3 (Conv2D)	(None, 46, 46, 128)	73856
conv4 (Conv2D)	(None, 44, 44, 128)	147584
maxpool2 (MaxPooling2D)	(None, 14, 14, 128)	0
conv5 (Conv2D)	(None, 12, 12, 256)	295168
conv6 (Conv2D)	(None, 10, 10, 256)	590080
maxpool3 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 1024)	2360320
dense_1 (Dense)	(None, 256)	262400
dropout (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 6)	1542
Total params: 3,769,670		
Trainable params: 3,769,670		
Non-trainable params: 0		

6개의 클래스 분류(학습)

다음은 인공신경망 네트워크를 구성하여 **이미지 분류**를 위한 **모델**을 만든다.

(예시) 3층 인공신경망 모델의 **입력층**, **은닉층**, **출력층**으로 **신호 전달하는 과정**



3층 인공신경망 모델

$(2, 3)$

$$\begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

$$(b_1^{(1)}, b_2^{(1)}, b_3^{(1)})$$

$(3, 2)$

$$\begin{pmatrix} w_{11}^{(2)} & w_{21}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} \\ w_{13}^{(2)} & w_{23}^{(2)} \end{pmatrix}$$

$$(b_1^{(2)}, b_2^{(2)})$$

$(2, 2)$

$$\begin{pmatrix} w_{11}^{(3)} & w_{21}^{(3)} \\ w_{12}^{(3)} & w_{22}^{(3)} \end{pmatrix}$$

$$(b_1^{(3)}, b_2^{(3)})$$

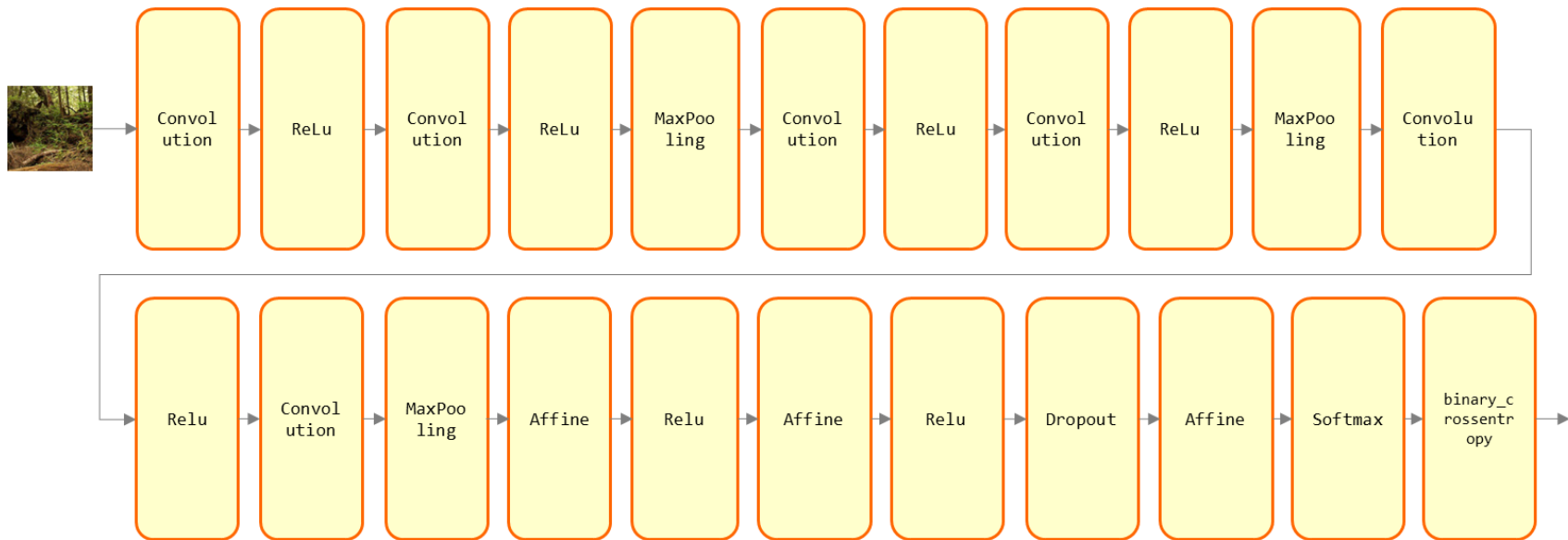
가중치

편향

⊕ 6개의 클래스 분류(학습)

다음은 인공신경망 네트워크를 구성하여 이미지 분류를 위한 모델을 만든다.

9층 인공신경망 모델의 계층구조로 이루어진 네트워크 구조



인공신경망 네트워크 구성

⊕ 6개의 클래스 분류(학습)

정의된 **인공신경망 모델**을 **학습**시켜 보자.

```
▶ # model training(모델을 학습시킵니다)
history = model.fit_generator(generator=train_generator,
                              steps_per_epoch=140,
                              epochs=40,
                              validation_data=test_generator,
                              validation_steps=30)
```

위 코드에서 모델을 **학습**시키기 위해 필요한 **하이퍼파라미터**를 **설정**하고
40 에폭 반복 학습을 수행함

6개의 클래스 분류(학습)

정의된 인공신경망 모델을 학습시켜 보자.

40 에폭 반복 학습을 진행한 결과

```
Epoch 19/40
140/140 [=====] - 114s 812ms/step - loss: 0.7089 - accuracy: 0.8392 - val_loss: 0.7262 - val_accuracy: 0.8373
Epoch 20/40
140/140 [=====] - 115s 819ms/step - loss: 0.6866 - accuracy: 0.8392 - val_loss: 0.6836 - val_accuracy: 0.8397
Epoch 21/40
140/140 [=====] - 114s 813ms/step - loss: 0.6763 - accuracy: 0.8390 - val_loss: 0.7198 - val_accuracy: 0.8237
Epoch 22/40
140/140 [=====] - 114s 817ms/step - loss: 0.6545 - accuracy: 0.8442 - val_loss: 0.7171 - val_accuracy: 0.8193
Epoch 23/40
140/140 [=====] - 113s 807ms/step - loss: 0.6401 - accuracy: 0.8457 - val_loss: 0.6406 - val_accuracy: 0.8390
Epoch 24/40
140/140 [=====] - 114s 812ms/step - loss: 0.6179 - accuracy: 0.8493 - val_loss: 0.6049 - val_accuracy: 0.8567
Epoch 25/40
140/140 [=====] - 114s 810ms/step - loss: 0.6054 - accuracy: 0.8510 - val_loss: 0.6077 - val_accuracy: 0.8537
Epoch 26/40
140/140 [=====] - 114s 811ms/step - loss: 0.5864 - accuracy: 0.8547 - val_loss: 0.6072 - val_accuracy: 0.8537
Epoch 27/40
140/140 [=====] - 113s 804ms/step - loss: 0.5550 - accuracy: 0.8644 - val_loss: 0.6016 - val_accuracy: 0.8500
Epoch 28/40
140/140 [=====] - 112s 801ms/step - loss: 0.5617 - accuracy: 0.8580 - val_loss: 0.5683 - val_accuracy: 0.8567
Epoch 29/40
140/140 [=====] - 114s 816ms/step - loss: 0.5451 - accuracy: 0.8629 - val_loss: 0.5913 - val_accuracy: 0.8537
Epoch 30/40
140/140 [=====] - 113s 810ms/step - loss: 0.5339 - accuracy: 0.8625 - val_loss: 0.5506 - val_accuracy: 0.8633
Epoch 31/40
140/140 [=====] - 113s 810ms/step - loss: 0.5061 - accuracy: 0.8713 - val_loss: 0.5605 - val_accuracy: 0.8547
Epoch 32/40
140/140 [=====] - 112s 802ms/step - loss: 0.5024 - accuracy: 0.8720 - val_loss: 0.5477 - val_accuracy: 0.8543
Epoch 33/40
140/140 [=====] - 113s 805ms/step - loss: 0.4846 - accuracy: 0.8727 - val_loss: 0.5717 - val_accuracy: 0.8473
Epoch 34/40
140/140 [=====] - 114s 812ms/step - loss: 0.4814 - accuracy: 0.8765 - val_loss: 0.6084 - val_accuracy: 0.8417
Epoch 35/40
140/140 [=====] - 114s 813ms/step - loss: 0.4818 - accuracy: 0.8747 - val_loss: 0.5829 - val_accuracy: 0.8367
Epoch 36/40
140/140 [=====] - 113s 809ms/step - loss: 0.4703 - accuracy: 0.8739 - val_loss: 0.5043 - val_accuracy: 0.8657
Epoch 37/40
140/140 [=====] - 114s 813ms/step - loss: 0.4563 - accuracy: 0.8810 - val_loss: 0.5619 - val_accuracy: 0.8457
Epoch 38/40
140/140 [=====] - 114s 814ms/step - loss: 0.4407 - accuracy: 0.8836 - val_loss: 0.4975 - val_accuracy: 0.8670
Epoch 39/40
140/140 [=====] - 113s 808ms/step - loss: 0.4396 - accuracy: 0.8824 - val_loss: 0.5005 - val_accuracy: 0.8663
Epoch 40/40
140/140 [=====] - 114s 811ms/step - loss: 0.4235 - accuracy: 0.8861 - val_loss: 0.5733 - val_accuracy: 0.8407
```

⊕ 6개의 클래스 분류(학습)

학습된 인공지능망 모델을 테스트 데이터 셋으로 평가한 결과이다.

```
▶ # model evaluate(모델을 평가해 봅니다)
score = model.evaluate(test_generator, steps=30, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1]*100)
```

```
↳ Test loss: 0.49203458428382874
    Test accuracy: 86.73333525657654
```

실행 결과 테스트 데이터 셋에 대한 정확도가 86.73% 나온 것을 볼 수 있음

⊕ 6개의 클래스 분류(학습)

학습된 인공지능망 모델에 대해 훈련 데이터 셋과 검증용 데이터 셋의 손실값(Loss), 정확도(Accuracy)의 추이를 그래프로 표현해 보자.

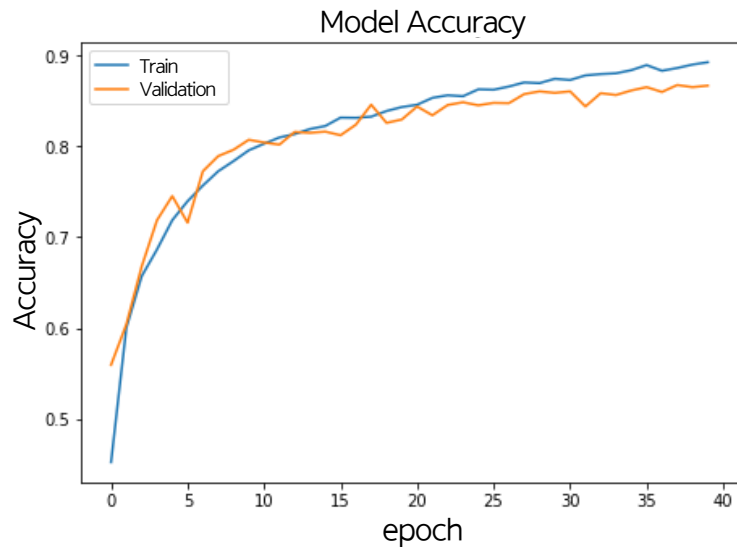
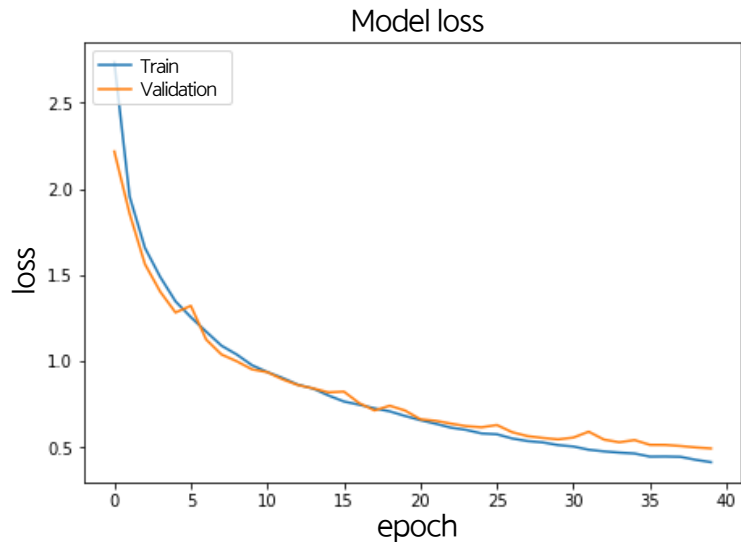
```
▶ plt.figure(figsize=(16,5))

# 만들어진 모델에 대해 train dataset과 validation dataset의 loss 를 그래프로 표현해 봅니다.
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')

# 만들어진 모델에 대해 train dataset과 validation dataset의 accuracy 를 그래프로 표현해 봅니다.
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')
```

6개의 클래스 분류(학습)

학습된 인공지능망 모델에 대해 훈련 데이터 셋과 검증용 데이터 셋의 손실값(Loss), 정확도(Accuracy)의 추이를 그래프로 표현해 보자.



- 훈련 데이터 셋의 경우 손실값이 점차 줄어 학습이 잘 진행되는 것으로 보임
- 훈련 데이터 셋의 경우 정확도가 점차 높아지고 있어 학습이 잘 진행되는 것으로 보임

⊕ 6개의 클래스 분류(학습)

학습한 인공지능망 모델을 H5 파일 형식으로 저장한다.



전체 모델을 H5 파일로 저장합니다

```
model.save('Intel_image_classification_model.h5')
```

```
print("Saved model to disk")
```



Saved model to disk

실행 결과 Intel_image_classification_model.h5 파일 이름으로
VM에 저장된 것을 알 수 있음

⊕ 6개의 클래스 분류(학습)

다음은 디렉토리(directory)에서 이미지 파일들을 읽어오는 함수 `get_images()`를 정의한다.

🕒 `get_images()` 함수는 지정한 디렉토리에서 25장의 이미지를 읽어옴

```
▶ # 디렉토리에서 이미지 파일들을 읽어오는 함수 정의
def get_images(directory):
    Images = []
    pred_img_cnt = 25 # 예측할 이미지를 몇개 불러 올 것인지 설정
    i = 0
    for image_file in os.listdir(directory): # 디렉토리로부터 파일을 하나하나 불러 온다
        i += 1
        if i > pred_img_cnt: break
        else:
            image = cv2.imread(directory+r'/' + image_file) # 불러온 이미지를 openCV 로 읽어 들인다
            image = cv2.resize(image, (150,150)) # 이미지 크기 재조정
            Images.append(image)

    return Images
```

⊕ 6개의 클래스 분류(학습)

다음은 예측에 사용할 이미지를 읽어오는 코드이다.

```
▶ # 예측 할 이미지를 불러온다  
pred_images = get_images('gdrive/My Drive/CV/Intel Image Classification/seg_pred/seg_pred')  
pred_images = np.array(pred_images) * 1./255  
pred_images.shape # (이미지수, 세로(H), 가로(W), 채널(C))
```

↳ (25, 150, 150, 3)

실행 결과 예측에 사용할 수 있는 (150, 150, 3) 형상의 이미지가 25장인 것을 알 수 있음

⊕ 6개의 클래스 분류(학습)

학습된 모델에 예측할 이미지로 분류 예측을 수행해 보자.

- 여기서는 25장 이미지를 분류 예측하고 그 결과를 prediction 객체에 저장함

```
# 예측할 이미지를 우리가 만든 모델을 통해 분류 예측해 봅니다.  
prediction = model.predict(pred_images, verbose=1)
```

```
1/1 [=====] - 1s 794ms/step
```

- 다음은 prediction 객체의 형상을 확인한 것이며, 형상은 (25, 6)인 것을 알 수 있음

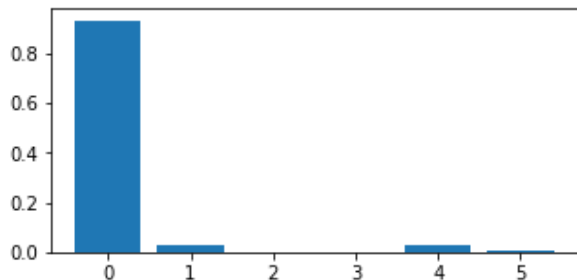
```
1 # 예측한 결과 객체 prediction shape 확인  
2 prediction.shape
```

```
(25, 6)
```

6개의 클래스 분류(학습)

학습된 모델에 예측할 이미지로 분류 예측을 수행해 보자.

- 다음은 prediction 객체의 예측결과를 확인한 것임
- 첫 번째 예측결과에서, 출력층의 활성화함수로 소프트맥스를 사용함으로 확률값이 출력된 것을 알 수 있음



➡ 출력층 활성화함수 : 소프트맥스

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5

- 첫 번째 이미지는 첫 번째 인덱스의 값이 가장 큰 것을 알 수 있음(즉, 빌딩으로 예측한 것임)

```
1 # 예측한 결과 객체 prediction 값 확인
2 prediction
3 #array([[7.01735616e-01, 2.82877952e-01, 6.54686708e-04, 1.84405053e-05, 9.91014822e-05, 1.46140968e-02],
4 #      [1.85270518e-01, 1.84901670e-04, 1.17759919e-05, 6.83128178e-07, 1.72170112e-05, 8.14514935e-01],
5 #      [1.34450952e-02, 2.03393437e-02, 2.66767085e-01, 5.47533333e-01, 1.33954734e-01, 1.79604013e-02],
6 #      [3.71958071e-04, 1.58283790e-03, 2.88972765e-01, 4.06801760e-01, 2.83311099e-01, 1.89595241e-02],
7 #      [3.52260517e-03, 9.94061353e-04, 6.41896841e-05, 9.30431634e-06, 4.10714256e-06, 9.95405674e-01],
8 #      [5.69388590e-07, 9.9997497e-01, 1.28447152e-06, 1.60010856e-07, 1.44987062e-08, 5.18240427e-07],
9 #      [9.77528274e-01, 2.22538393e-02, 2.75015427e-05, 1.55774799e-07, 1.07141068e-06, 1.89180617e-04],
10 #      [2.17310160e-01, 3.41707258e-03, 5.60939405e-03, 1.14200008e-03, 1.89870608e-03, 7.70622671e-01],
11 #      [2.98322109e-03, 4.10983717e-04, 2.44892053e-02, 4.61837620e-01, 5.05038023e-01, 5.24095725e-03],
```

⊕ 6개의 클래스 분류(학습)

다음은 학습된 모델이 예측한 25장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화해 보자.

예측결과 값은 확률값이고, 확률값이 가장 큰 인덱스 위치가 모델이 예측한 결과임

```
# 예측한 결과는 아래와 같습니다. 이미지의 위에는 class label을 이미지 아래에는 확률을 나타냅니다.

fig = plt.figure(figsize=(30, 30))
outer = gridspec.GridSpec(5, 5, wspace=0.2, hspace=0.2)

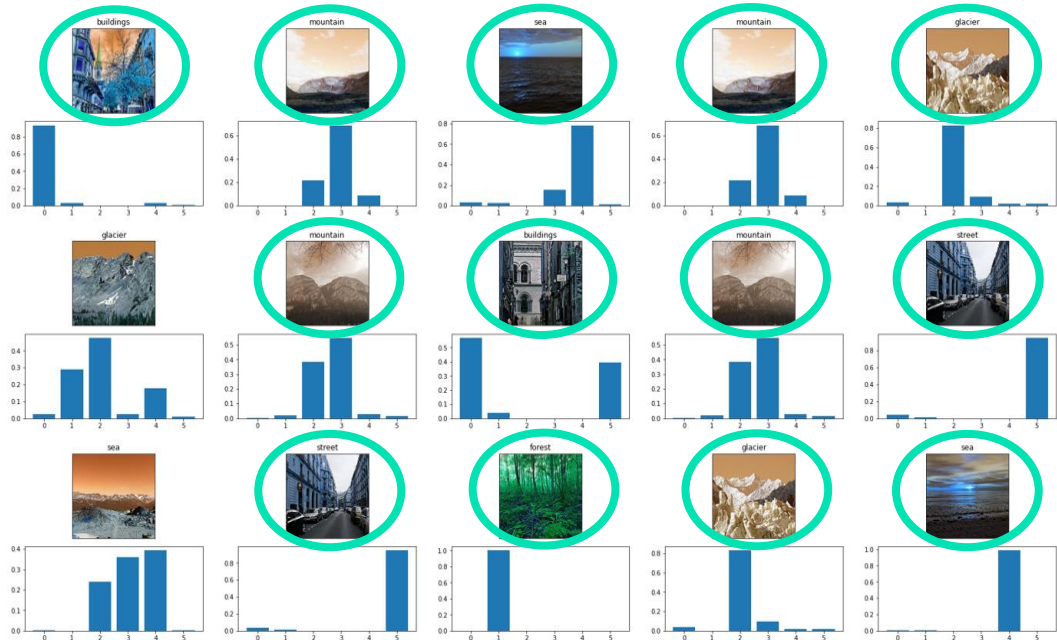
for i in range(25):
    inner = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=outer[i], wspace=0.1, hspace=0.1)
    rnd_number = randint(0, len(pred_images)-1)
    pred_image = np.array([pred_images[rnd_number]])

    pred_class = findKey(train_generator.class_indices, np.argmax(prediction[rnd_number])) # 이미지의 label name 을 찾아웁니다.
    pred_prob = prediction[rnd_number].reshape(6) # 예측된 결과(확률값)를 1차원 배열(6칸)로 reshape 합니다.
    for j in range(2):
        if (j%2) == 0:
            # 처음에는 이미지와 title(label name) 을 보여줍니다.
            ax = plt.Subplot(fig, inner[j])
            ax.imshow(pred_image[0])
            ax.set_title(pred_class)
            ax.set_xticks([])
            ax.set_yticks([])
            fig.add_subplot(ax)
        else:
            # 그다음은 확률값을 그래프 형태로 보여줍니다.
            ax = plt.Subplot(fig, inner[j])
            ax.bar([0,1,2,3,4,5], pred_prob)
            fig.add_subplot(ax)

fig.show()
```

다음은 학습된 모델이 예측한 25장 이미지의 예측결과(predictions 객체에 저장된 결과)를 시각화해 보자.

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5

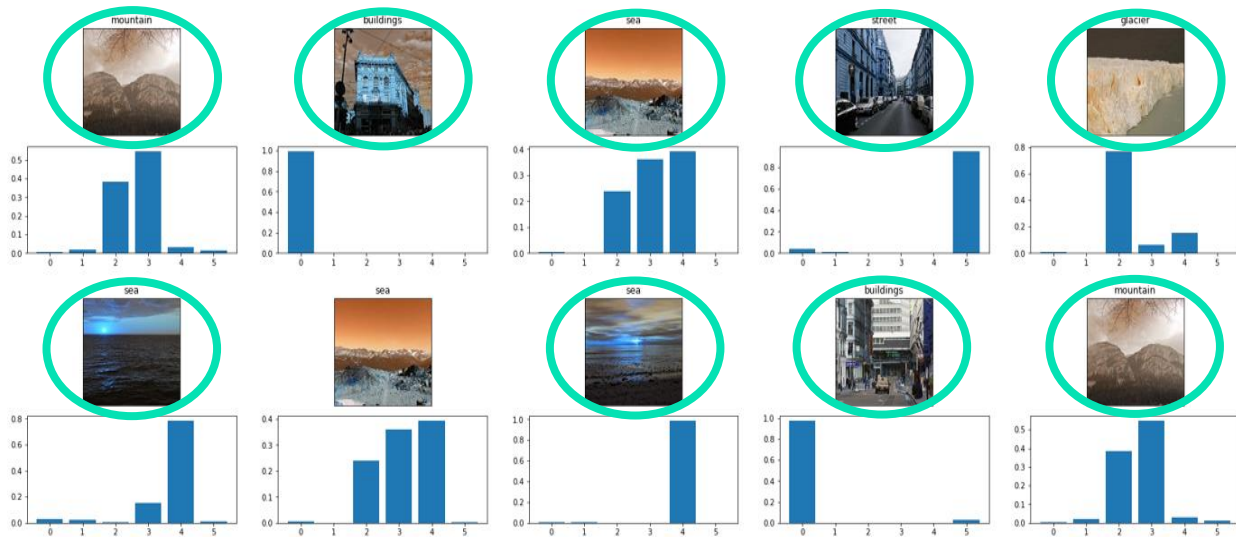


6개의 클래스 분류(학습)

다음은 학습된 모델이 예측한 25장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화해 보자.

예측결과

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5



정답 : 22 / 오답 : 3